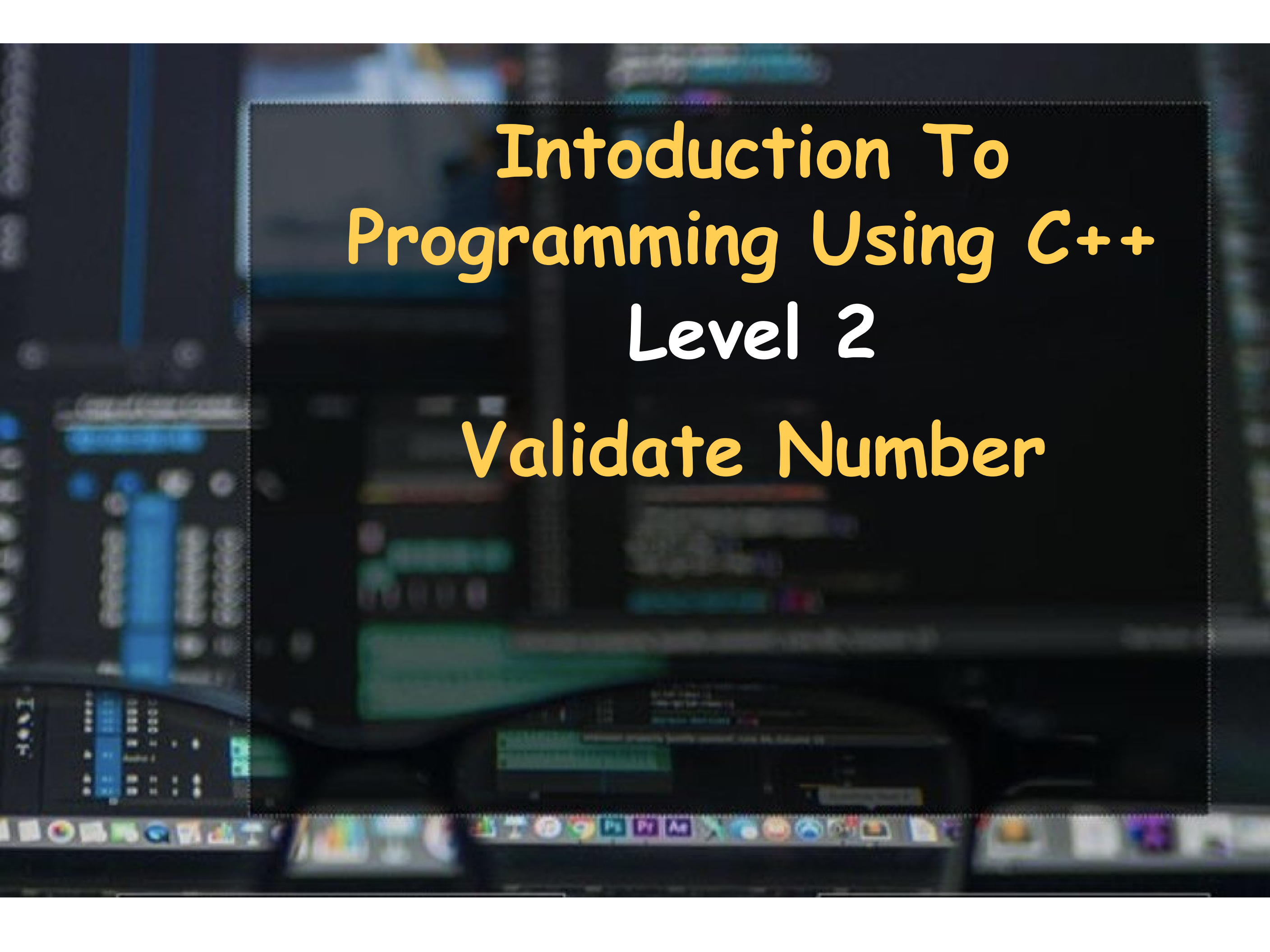
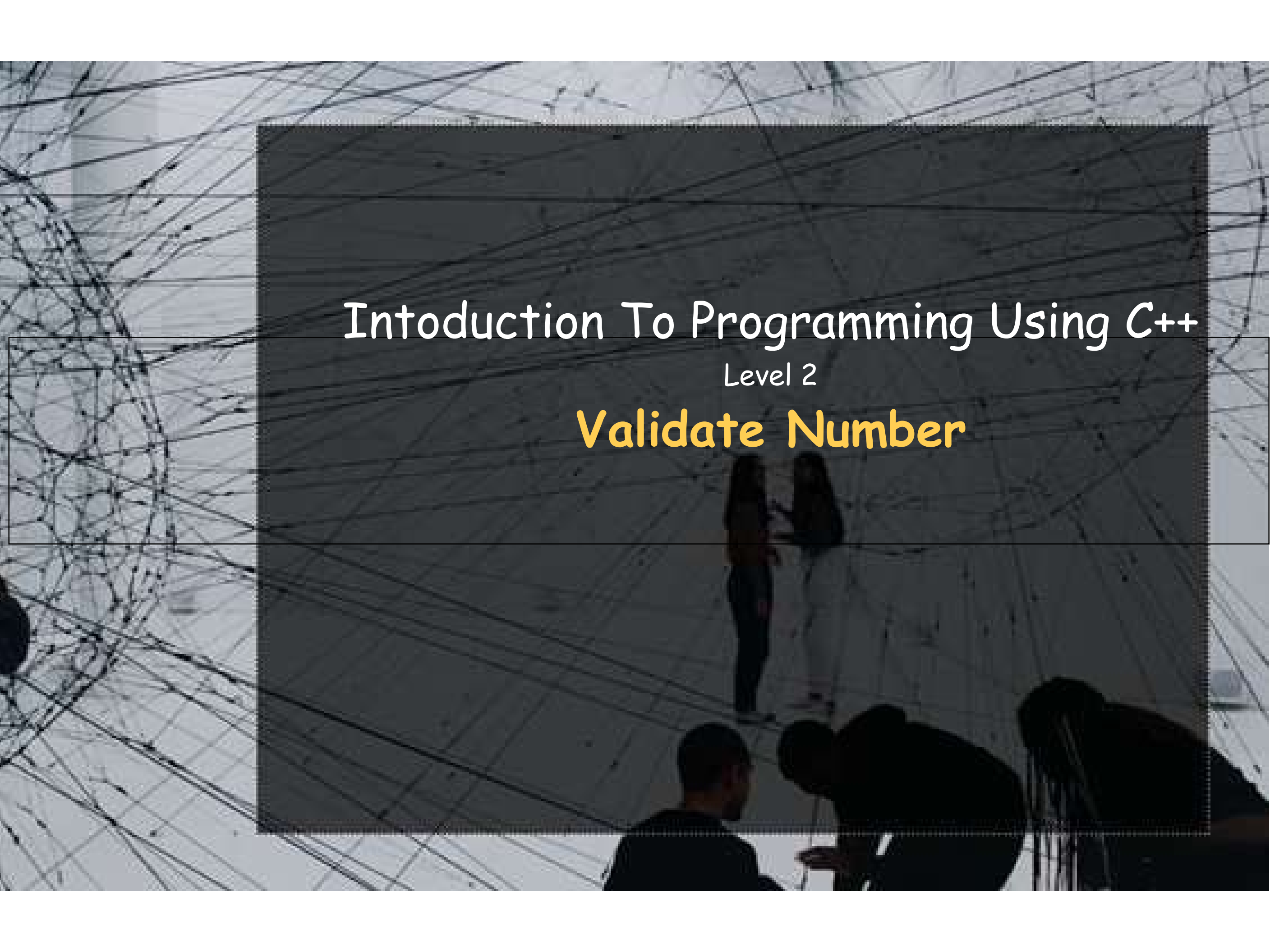


The background of the slide is a blurred image of a computer monitor. On the left side, a calculator application is visible, showing a numeric keypad and some text. The main area of the monitor displays what appears to be a code editor with lines of text in various colors (green, blue, white) on a dark background, typical of a programming IDE. At the bottom of the monitor, a taskbar with several application icons is visible.

Introduction To Programming Using C++ Level 2

Validate Number



Introduction To Programming Using C++

Level 2

Validate Number

Definition

- ◆ Number Validation is the process of checking whether the value entered by the user is valid for the expected Variable Type.
- ◆ In C++, `cin` reads input from the Input Buffer and attempts to convert it to the type of the Variable.

Explanation

- If the user enters a valid value, ``cin`` successfully stores the value in the Variable.
- If the user enters an invalid value, ``cin`` enters an Error State.
- We can check this using ``cin.fail()``.
- When an input error occurs, ``cin.fail()`` returns ``true``.
- Before reading new input, we need to clear the Error State and remove the invalid input from the Input Buffer.

Procedures

1. Read the user's input using `cin`.
2. Check whether the input is valid.
3. Use `cin.fail()` to detect an input error.
4. If `cin.fail()` is `true`, clear the Error State using `cin.clear()`.
5. Remove the invalid input from the Input Buffer using `cin.ignore()`.
6. Ask the user to enter the value again.

💡 Example

- ◆ Suppose we want the user to enter a Number:

```
int Number;  
cout << "Enter a number: ";  
cin >> Number;  
if (cin.fail())  
{  
    cin.clear();  
    cin.ignore(1000, '\n');  
    cout << "Invalid Number!";  
}
```

- ◆ If the user enters:
25

- ◆ The input is valid and `Number` receives the value `25`.

- ◆ If the user enters:
Hello

- ◆ The input is invalid, so `cin.fail()` becomes `true`.

- ◆ We can then use:

```
cin.clear();  
cin.ignore(1000, '\n');
```

Note

- ◆ `cin.clear()` and `cin.ignore()` do two different jobs.
- ◆ `cin.clear()` clears the Error State.
- ◆ `cin.ignore(1000, '\n')` removes unwanted characters from the Input Buffer.

⊘ Warning

- ◆ Calling only `cin.clear()` is not enough when invalid characters remain in the Input Buffer.
- ◆ If the invalid input is not removed, the next `cin` operation may encounter the same invalid characters again.



Comparison

▨ Valid Input

- ✓ The input can be converted to the expected Variable Type.
- ✓ `cin.fail()` remains `false`.
- ✓ The Variable receives the entered value.

▨ Invalid Input

- ✓ The input cannot be converted to the expected Variable Type.
- ✓ `cin.fail()` becomes `true`.
- ✓ We need to clear the Error State and handle the invalid input.

✦ Summary

- ✓ ``cin`` reads input from the Input Buffer and attempts to convert it to the expected type.
- ✓ ``cin.fail()`` checks whether an input error occurred.
- ✓ ``cin.clear()`` clears the Error State.
- ✓ ``cin.ignore()`` removes unwanted input from the Input Buffer.

Thank you 😊